

A Project Report On

Sports performance analysis

Submitted in partial fulfillment of the requirement for the
award of the degree

MASTER OF SCIENCE (DATA SCIENCE)

from

Marwadi University

Academic Year 2025 – 26

Nevil Aghera (92500584012)

Jenil Kothiya (92500584026)

Internal Guide

Gehna Sachdeva



Marwadi
University
Marwadi Chandarana Group



Rajkot-Morbi Road, At & PO : Gauridad, Rajkot 360 003. Gujarat. India.



Marwadi
University
Marwadi Chandarana Group



Faculty of Computer Applications (FoCA)

Certificate

**This is to certify that the project work entitled
Heart Failure Prediction Using Logistic Regression
submitted in partial fulfillment of the requirement for
the award of the degree of**

**Master of Science (Data Science)
of the**

Marwadi University

**is a result of the bonafide work carried out by
Nevil Aghera (92500584012)
Jenil Kothiya (92500584026)**

during the academic year 2025 – 2026

Gehna Sachdeva
Faculty Guide

Dr. Sunil Bajaja
HOD

Dr. Rizvan Khan
Dean

DECLARATION

We hereby declare that this project work entitled **Cricket Player Performance** is a record done by us.

We also declare that the matter embodied in this project is genuine work done by us and has not been submitted whether to this University or to any other University / Institute for the fulfillment of the requirement of any course of study.

Place: Rajkot

Date:

Nevil Aghera (92500584012)
Jenil Kothiya (92500584026)

Signature: _____
Signature: _____

ACKNOWLEDGEMENT

It is indeed a great pleasure to express our thanks and gratitude to all those who helped us. No serious and lasting achievement or success one can ever achieve without the help of friendly guidance and co-operation of so many people involved in the work.

We are very thankful to our guide **Gehna Sachdeva**, the person who makes us to follow the right steps during our project work. We express our deep sense of gratitude to for his guidance, suggestions and expertise at every stage. A part from that his valuable and expertise suggestion during documentation of our report indeed help us a lot.

Thanks to our friend and colleague who have been a source of inspiration and motivation that helped to us during our project work.

We are heartily thankful to the Dean of our department **Dr. Rizvan Khan** for giving us an opportunity to work over this project and for their end-less and great support to all other people who directly or indirectly supported and help us to fulfil our task.

Nevil Aghera (92500584012)
Jenil Kothiya (92500584026)

Signature: _____
Signature: _____

CONTENTS

Chapters	Particulars	Page No.
1	Introduction 1.1. Objective of the New System 1.2. Problem Definition 1.3. Core Components 1.4. Project Profile 1.5. Assumptions and Constraints 1.6. Advantages and Limitations of the Proposed System	6
2	Requirement Determination & Analysis 2.1. Requirement Determination 2.2. Targeted Users 2.3. Tool details (Python / PowerBI/ Tableau) 2.4. Library description (Details on various libraries / packages used)	7
3	System Design 3.1. Flowchart / Algorithm with steps 3.2. Dataset Design 3.3. Details on preprocessing steps applied	8 9 9
4	Development 4.1 Source code / Output 4.2. Test Reports	10
5	Proposed Enhancements	23
6	Conclusion	23
7	Bibliography	23

1. Introduction:

1.1 Objective of the New System

- The objective of this system is to analyze cricket player performance data and generate meaningful insights such as batting form, strike rate trends, and impact scores, helping coaches, analysts, and selectors make data-driven decisions.

1.2 Problem Definition

- Currently, player performance evaluation is done manually using scattered scorecards, which is time-consuming and prone to inconsistency. There is a need for a centralized system that automatically processes match data and provides accurate, quick performance analysis.

1.3 Core Components

- The system consists of a data collection module (match records), a processing/analysis engine (statistics, impact score calculation), a database to store player and match data, and a reporting/visualization module to present results.

1.4 Project Profile

- The project is a data-driven Cricket Player Performance Analysis System built to process historical match data (runs, wickets, strike rate, economy rate, etc.) and generate performance summaries and predictive insights for players across match formats (T20, ODI, Test).

1.5 Assumptions and Constraints

- It is assumed that input data is accurate and complete. The system is constrained by the availability of historical data and does not account for real-time weather or pitch conditions during analysis.

1.6 Advantages and Limitations of the Proposed System

- **Advantages:** Fast and accurate analysis, reduces manual effort, supports better decision-making.
- **Limitations:** Depends on data quality; does not factor in psychological or situational match pressure on players.

2. Requirement Determination & Analysis:

2.1 Requirement Determination

- Historical match data required (runs, balls faced, fours, sixes, wickets, economy rate, strike rate, match result).
- Data cleaning and preprocessing to handle missing/incorrect values.
- Exploratory Data Analysis (EDA) to understand patterns and trends.
- Feature engineering to create relevant input variables for the model.
- Build ML model to predict player performance/impact score.
- Visualize results for easy interpretation by non-technical users.

2.2 Targeted Users

- Cricket team analysts
- Coaches and selectors
- Sports journalists
- Fantasy league players
- Sports management organizations

2.3 Tool Details (Python / Power BI / Tableau)

- Python: Data preprocessing, EDA, and ML model building/training.
- Power BI / Tableau: Interactive dashboards for visualizing player stats and trends.
- Jupyter Notebook: Development environment for coding and testing.

2.4 Library Description

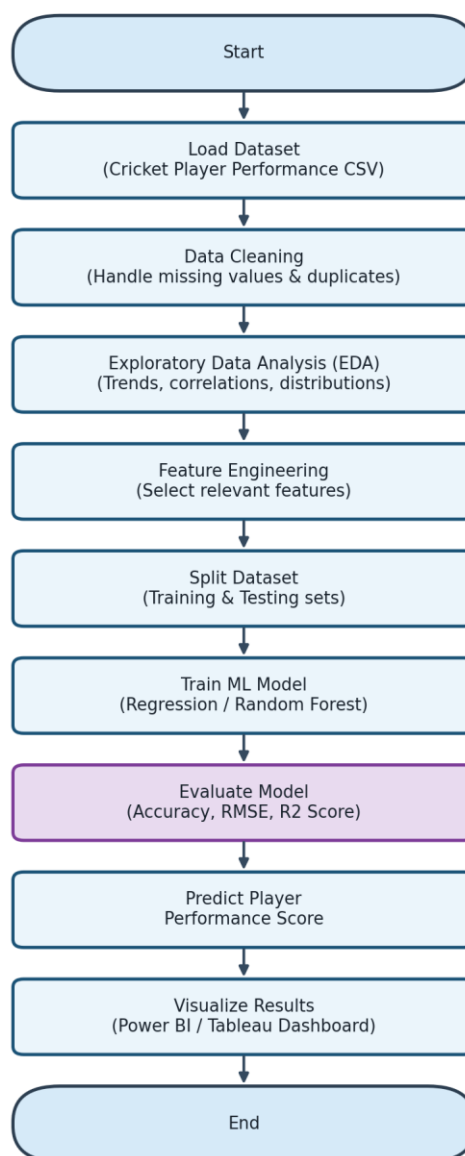
- Pandas: Data loading, cleaning, and manipulation.
- NumPy: Numerical computations on dataset.
- Matplotlib / Seaborn: Data visualization (graphs, charts, correlations).
- Scikit-learn: ML model building (Regression/Random Forest) and evaluation.
- Jupyter Notebook: Interactive coding and step-by-step analysis

3. System Design:

3.1 Flowchart / Algorithm with steps:

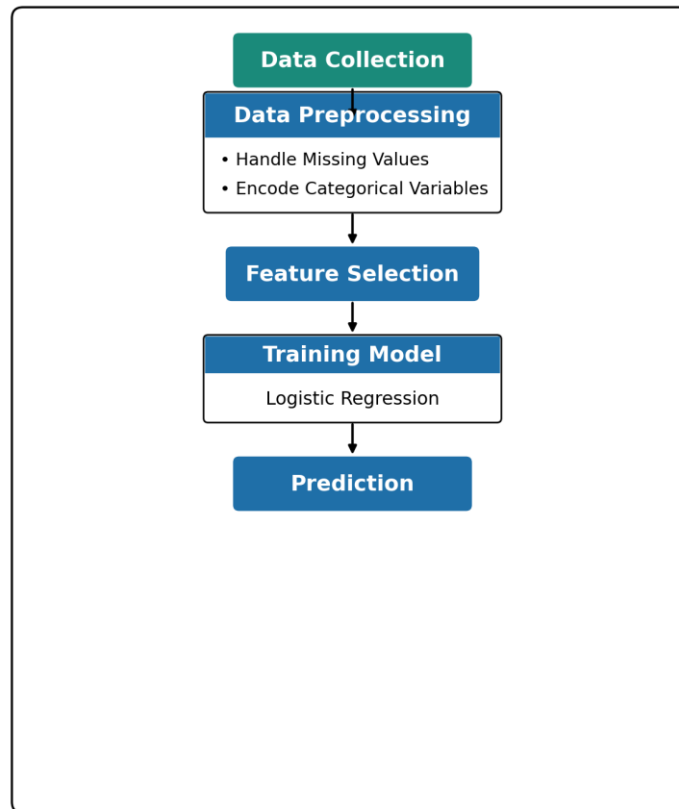
- Step 1: Load dataset
- Step 2: Replace missing values using SimpleImputer
- Step 3: Encode categorical values using LabelEncoder
- Step 4: Scale numerical values using MinMaxScaler
- Step 5: Split data into training and test sets
- Step 6: Train models using different algorithms
- Step 7: Evaluate model performance using metrics
- Step 8: Visualize data using heatmap, scatter, boxplot, and histogram

➤ **Flowchart:**



3.2 Dataset Design:

Cricket Match Outcome Prediction Using Logistic Regression



The dataset includes 40,000 real-world style samples of individual player performances across international cricket matches, with attributes such as runs scored, balls faced, boundaries (fours and sixes), wickets taken, economy rate, strike rate, catches, and target score. The target variable is "Match_Result", which classifies the outcome of the match as Win, Loss, or Tie.

3.3 Details on preprocessing steps applied:

- **Missing Values:** Dataset checked for nulls; none were found. As a precaution, SimpleImputer (mean for numerical, most frequent for categorical) is applied in the pipeline.
- **Categorical Encoding:** LabelEncoder applied to all object type columns (Player_Name, Team, Opponent, Match_Type, Match_Result).
- **Feature Scaling:** MinMaxScaler used on numerical columns (Runs, Balls_Faced, Fours, Sixes, Wickets, Economy_Rate, Player_Impact_Score, Strike_Rate, Catches, Target_Score) to normalize them between 0 and 1.
- **Date Handling:** The Date column is parsed and decomposed into Year, Month, and Day features before dropping the original column.

4. Development:

4.1 Source Code / Output

➤ Source Code :

Cricket Player Performance

1. Import Tools

```
"""
```

```
# Commented out IPython magic to ensure Python compatibility.
```

```
import pandas as pd
```

```
import numpy as np
```

```
import seaborn as sns
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
```

```
from sklearn.impute import SimpleImputer
```

```
from sklearn.svm import SVC
```

```
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
```

```
# %matplotlib inline
```

```
pd.set_option('display.max_columns', None)
```

""""## 2. Import Data""""

```
df = pd.read_csv('cricket_player_performance.csv')
```

```
df
```

""""## 3. HBIS — Head, Basic Info & Shape""""

```
# Head - first 5 rows
```

```
df.head()
```

```
# Basic Info - column dtypes & non-null counts
```

```
df.info()
```

```
# Shape - rows x columns
```

```
print("Shape:", df.shape)
```

```
# Statistical Summary
```

```
df.describe()
```

"""## 4. Train Data — Clean, EDA, Label Encode, Split

4.1 Data Cleaning (Missing Values & Duplicates)

```
# Replace placeholder characters with NaN
df.replace('?', np.nan, inplace=True)

# Check missing values
print("Missing values per column:\n", df.isnull().sum())
print("\nTotal missing values:", df.isnull().sum().sum())

# Check duplicates
print("\nDuplicate rows:", df.duplicated().sum())
df.drop_duplicates(inplace=True)

# Separate numeric and categorical columns
numerical_cols = df.select_dtypes(include=['number']).columns
categorical_cols = df.select_dtypes(include=['object', 'string']).columns

# Impute (safety step, applies only if missing values exist)
if df[numerical_cols].isnull().sum().sum() > 0:
    df[numerical_cols] = SimpleImputer(strategy='mean').fit_transform(df[numerical_cols])

if df[categorical_cols].isnull().sum().sum() > 0:
    df[categorical_cols] = SimpleImputer(strategy='most_frequent').fit_transform(df[categorical_cols])

print("Remaining missing values:", df.isnull().sum().sum())
```

"""### 4.2 Feature Engineering — Extract Date Parts"""

```
df['Date'] = pd.to_datetime(df['Date'])
df['Match_Year'] = df['Date'].dt.year
df['Match_Month'] = df['Date'].dt.month
df.drop(columns=['Date'], inplace=True)
df.head()
```

"""### 4.3 Exploratory Data Analysis (EDA)"""

```
# Target class distribution
print(df['Match_Result'].value_counts())

plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='Match_Result')
plt.title("Match Result Distribution")
plt.show()

# Match type distribution
plt.figure(figsize=(6, 4))
sns.countplot(data=df, x='Match_Type')
plt.title("Match Type Distribution")
plt.show()
```

```
# Correlation among numeric features
plt.figure(figsize=(6, 5))
sns.heatmap(df.select_dtypes(include=['number']).corr(), annot=True, cmap="coolwarm", fmt=".2f",
linewidths=0.5)
plt.title("Feature Correlation Heatmap")
plt.show()
```

"""### 4.4 Label Encoding (Categorical Columns)"""

```
categorical_cols = df.select_dtypes(include=['object', 'string']).columns
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

print("Match_Result classes:", dict(enumerate(label_encoders['Match_Result'].classes_)))
df.head()
```

"""### 4.5 Feature Scaling"""

```
target = 'Match_Result'
numerical_cols = df.select_dtypes(include=['number']).columns
scale_cols = [c for c in numerical_cols if c != target]

scaler = MinMaxScaler()
df[scale_cols] = scaler.fit_transform(df[scale_cols])
df.head()
```

"""### 4.6 Train-Test Split"""

```
features = df.columns[df.columns != target]
X = df[features]
y = df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=0)

print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
```

"""## 5. Fit Algorithms on Train Dataset & Evaluate One by One

5.1 SVM (Support Vector Machine)

```
"""
```

```
svm_model = SVC(kernel='rbf', C=1, gamma='scale')
svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)

print("Confusion Matrix (SVM):\n", confusion_matrix(y_test, y_pred_svm))
print("\nAccuracy (SVM):", round(accuracy_score(y_test, y_pred_svm) * 100, 2), "%")
print("\nClassification Report (SVM):\n", classification_report(y_test, y_pred_svm))
```

5.2 Naive Bayes (NB)#####

```
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)
y_pred_nb = nb_model.predict(X_test)

print("Confusion Matrix (Naive Bayes):\n", confusion_matrix(y_test, y_pred_nb))
print("\nAccuracy (Naive Bayes):", round(accuracy_score(y_test, y_pred_nb) * 100, 2), "%")
print("\nClassification Report (Naive Bayes):\n", classification_report(y_test, y_pred_nb))
```

5.3 K-Nearest Neighbors (KNN)#####

```
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
y_pred_knn = knn_model.predict(X_test)

print("Confusion Matrix (KNN):\n", confusion_matrix(y_test, y_pred_knn))
print("\nAccuracy (KNN):", round(accuracy_score(y_test, y_pred_knn) * 100, 2), "%")
print("\nClassification Report (KNN):\n", classification_report(y_test, y_pred_knn))
```

5.4 Decision Tree#####

```
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)

print("Confusion Matrix (Decision Tree):\n", confusion_matrix(y_test, y_pred_dt))
print("\nAccuracy (Decision Tree):", round(accuracy_score(y_test, y_pred_dt) * 100, 2), "%")
print("\nClassification Report (Decision Tree):\n", classification_report(y_test, y_pred_dt))
```

5.5 Random Forest#####

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

print("Confusion Matrix (Random Forest):\n", confusion_matrix(y_test, y_pred_rf))
print("\nAccuracy (Random Forest):", round(accuracy_score(y_test, y_pred_rf) * 100, 2), "%")
print("\nClassification Report (Random Forest):\n", classification_report(y_test, y_pred_rf))
```

6. Model Comparison (Evaluation Summary)#####

```
results = {
'SVM': accuracy_score(y_test, y_pred_svm),
'Naive Bayes': accuracy_score(y_test, y_pred_nb),
'KNN': accuracy_score(y_test, y_pred_knn),
'Decision Tree': accuracy_score(y_test, y_pred_dt),
'Random Forest': accuracy_score(y_test, y_pred_rf),
}

comparison_df = pd.DataFrame(list(results.items()), columns=['Model', 'Accuracy'])
comparison_df['Accuracy (%)'] = (comparison_df['Accuracy'] * 100).round(2)
```

```
comparison_df = comparison_df.sort_values('Accuracy (%)', ascending=False)
comparison_df
```

```
plt.figure(figsize=(9, 5))
sns.barplot(data=comparison_df, x='Accuracy (%)', y='Model', hue='Model', palette='mako', legend=False)
plt.title("Model Accuracy Comparison")
plt.xlabel("Accuracy (%)")
plt.tight_layout()
plt.show()
```

"""## 7. Prediction

Best-performing model (highest accuracy) use karke test set ke pehle 10 samples ki prediction actual values ke saath compare kar rahe hain.

"""

```
best_model_name = comparison_df.iloc[0]['Model']
model_lookup = {
'SVM': svm_model,
'Naive Bayes': nb_model,
'KNN': knn_model,
'Decision Tree': dt_model,
'Random Forest': rf_model,
}
best_model = model_lookup[best_model_name]
print(f'Best Model: {best_model_name}')
```

```
y_pred_best = best_model.predict(X_test)
```

```
result_table = X_test.copy()
result_table['Actual_Match_Result'] = label_encoders['Match_Result'].inverse_transform(y_test)
result_table['Predicted_Match_Result'] = label_encoders['Match_Result'].inverse_transform(y_pred_best)
```

```
result_table[['Actual_Match_Result', 'Predicted_Match_Result']].head(10)
```

"""## 8. Visualizations (Matplotlib)"""

Scatter Plot - Runs vs Strike Rate colored by Match Result

```
plt.figure(figsize=(8, 6))
sns.scatterplot(data=df, x='Runs', y='Strike_Rate', hue='Match_Result', palette='Set1', alpha=0.6)
plt.title("Scatter Plot: Runs vs Strike Rate (by Match Result)")
plt.show()
```

Histograms of Key Numerical Attributes (Grid Layout)

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
hist_cols = ['Runs', 'Balls_Faced', 'Strike_Rate', 'Economy_Rate', 'Player_Impact_Score']
```

```
colors = ['red', 'green', 'blue', 'orange', 'purple']
```

```
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.flatten()
```

```
for i, col in enumerate(hist_cols):
sns.histplot(
```

```

df[col],
bins=30,
kde=True,
ax=axes[i],
color=colors[i],
edgecolor='black',
alpha=0.8
)
axes[i].set_title(f'Histogram of {col}', fontsize=11)
axes[i].grid(True, linestyle='--', alpha=0.5)

```

```
axes[5].axis('off')
```

```

plt.tight_layout()
plt.show()

```

Box Plot - Match_Result distribution

```

plt.figure(figsize=(6, 4))
sns.boxplot(y=df['Match_Result'], color='skyblue')
plt.title("Box Plot of Match_Result (encoded)", fontsize=14)
plt.ylabel("Match_Result")
plt.tight_layout()
plt.show()

```

Box Plots without Outliers - Key Numeric Stats

```

numerical_columns = ['Runs', 'Balls_Faced', 'Strike_Rate', 'Player_Impact_Score', 'Target_Score']
fig, axes = plt.subplots(nrows=1, ncols=5, figsize=(18, 5))
for idx, col in enumerate(numerical_columns):
    data = df[col].dropna()
    axes[idx].boxplot(data, showfliers=False)
    axes[idx].set_title(col)
plt.tight_layout()
plt.show()

```

Confusion Matrix Heatmaps for all models

```

fig, axes = plt.subplots(2, 3, figsize=(16, 9))
axes = axes.flatten()

preds = {
'SVM': y_pred_svm,
'Naive Bayes': y_pred_nb,
'KNN': y_pred_knn,
'Decision Tree': y_pred_dt,
'Random Forest': y_pred_rf,
}

for i, (name, pred) in enumerate(preds.items()):
    cm = confusion_matrix(y_test, pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[i], cbar=False)
    axes[i].set_title(name)
    axes[i].set_xlabel("Predicted")
    axes[i].set_ylabel("Actual")

axes[5].axis('off')
plt.tight_layout()
plt.show()

```

➤ Output :

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 40000 entries, 0 to 39999
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Date                  40000 non-null object
1   Player_Name          40000 non-null object
2   Team                  40000 non-null object
3   Opponent              40000 non-null object
4   Runs                  40000 non-null int64
5   Balls_Faced           40000 non-null int64
6   Fours                  40000 non-null int64
7   Sixes                  40000 non-null int64
8   Wickets                40000 non-null int64
9   Economy_Rate          40000 non-null float64
10  Match_Result           40000 non-null object
11  Match_Type             40000 non-null object
12  Player_Impact_Score    40000 non-null float64
13  Strike_Rate            40000 non-null float64
14  Catches                40000 non-null int64
15  Target_Score           40000 non-null int64
dtypes: float64(3), int64(7), object(6)
memory usage: 4.9+ MB
```

Shape: (40000, 16)

Missing values per column:

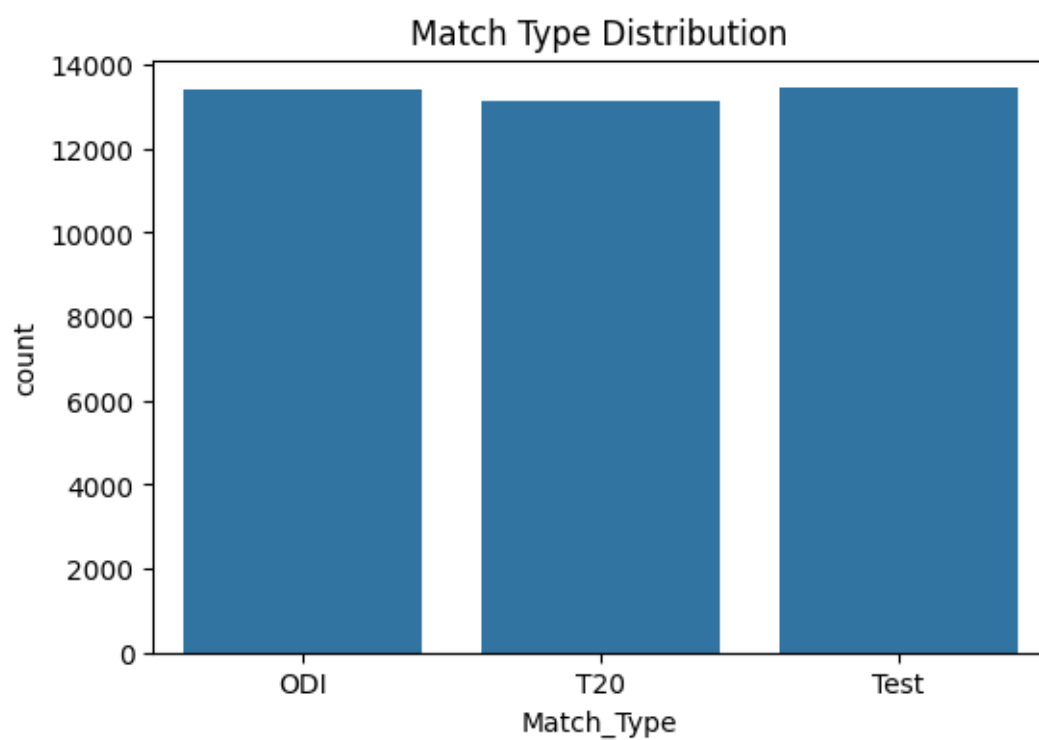
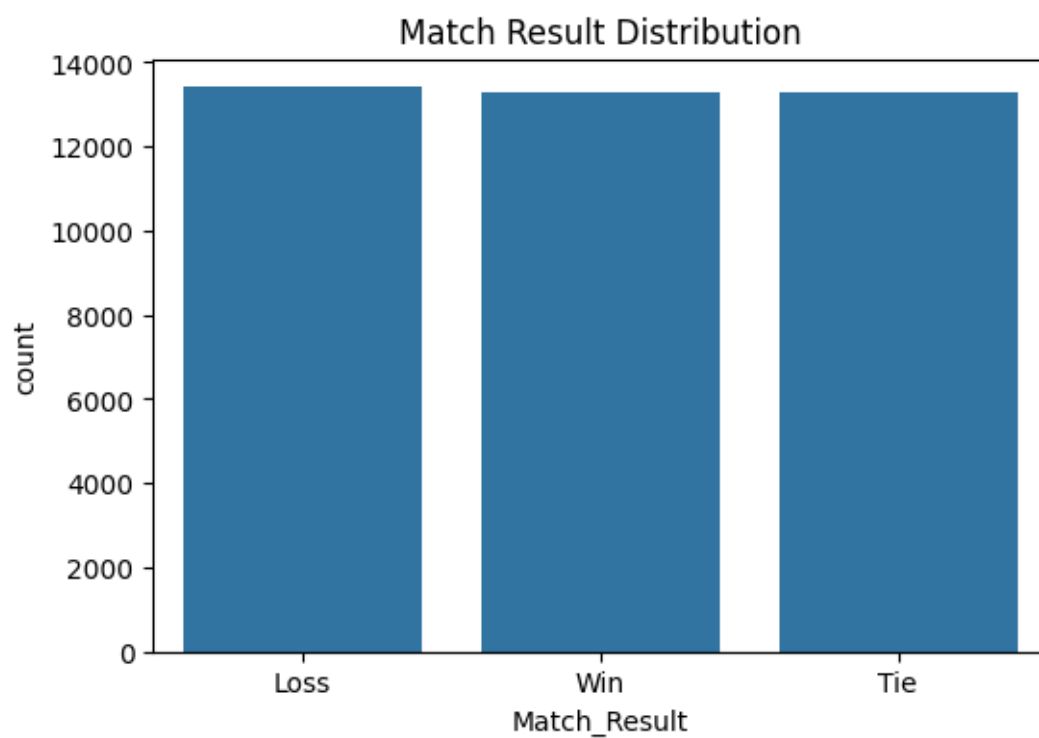
```
Date          0
Player_Name    0
Team           0
Opponent       0
Runs           0
Balls_Faced    0
Fours          0
Sixes          0
Wickets        0
Economy_Rate   0
Match_Result   0
Match_Type     0
Player_Impact_Score  0
Strike_Rate    0
Catches        0
Target_Score   0
dtype: int64
```

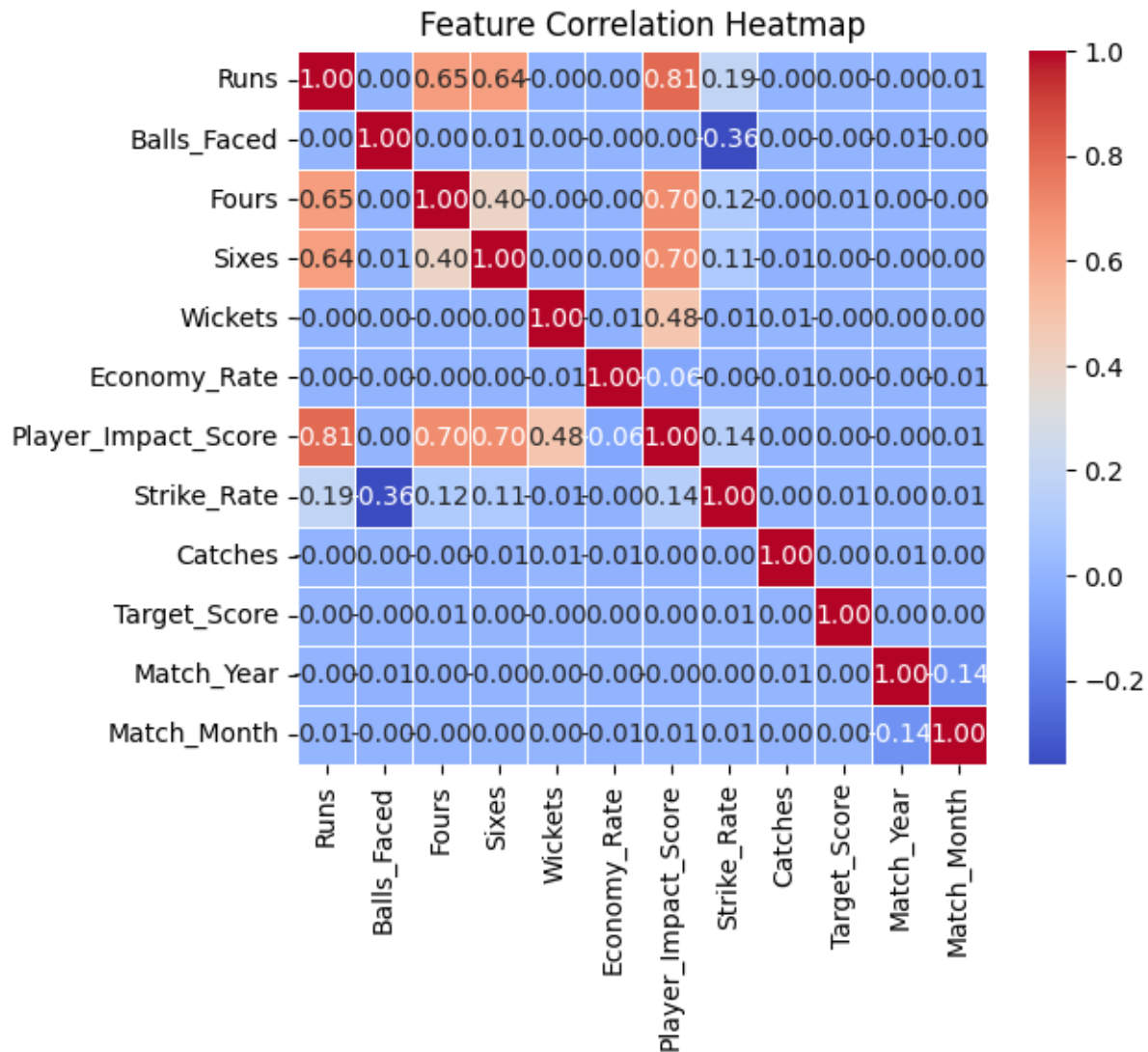
Total missing values: 0

Duplicate rows: 0

Remaining missing values: 0

Match_Result
Loss 13415
Tie 13304
Win 13281
Name: count, dtype: int64





Match_Result classes: {0: 'Loss', 1: 'Tie', 2: 'Win'}

Training set: (28000, 16)

Testing set: (12000, 16)

Confusion Matrix (SVM):

[[1634 1066 1299]

[1650 1119 1309]

[1608 1016 1299]]

Accuracy (SVM): 33.77 %

Classification Report (SVM):

	precision	recall	f1-score	support
--	-----------	--------	----------	---------

0	0.33	0.41	0.37	3999
---	------	------	------	------

1	0.35	0.27	0.31	4078
---	------	------	------	------

2	0.33	0.33	0.33	3923
---	------	------	------	------

accuracy			0.34	12000
macro avg	0.34	0.34	0.34	12000
weighted avg	0.34	0.34	0.34	12000

Confusion Matrix (Naive Bayes):

[[502 2260 1237]

[488 2353 1237]
[501 2256 1166]]

Accuracy (Naive Bayes): 33.51 %

Classification Report (Naive Bayes):

	precision	recall	f1-score	support
0	0.34	0.13	0.18	3999
1	0.34	0.58	0.43	4078
2	0.32	0.30	0.31	3923
accuracy		0.34		12000
macro avg	0.33	0.33	0.31	12000
weighted avg	0.33	0.34	0.31	12000

Confusion Matrix (KNN):

[[1828 1339 832]
[1833 1358 887]
[1816 1303 804]]

Accuracy (KNN): 33.25 %

Classification Report (KNN):

	precision	recall	f1-score	support
0	0.33	0.46	0.39	3999
1	0.34	0.33	0.34	4078
2	0.32	0.20	0.25	3923
accuracy		0.33		12000
macro avg	0.33	0.33	0.32	12000
weighted avg	0.33	0.33	0.32	12000

Confusion Matrix (Decision Tree):

[[1363 1308 1328]
[1357 1374 1347]
[1294 1319 1310]]

Accuracy (Decision Tree): 33.73 %

Classification Report (Decision Tree):

	precision	recall	f1-score	support
0	0.34	0.34	0.34	3999
1	0.34	0.34	0.34	4078
2	0.33	0.33	0.33	3923
accuracy		0.34		12000
macro avg	0.34	0.34	0.34	12000
weighted avg	0.34	0.34	0.34	12000

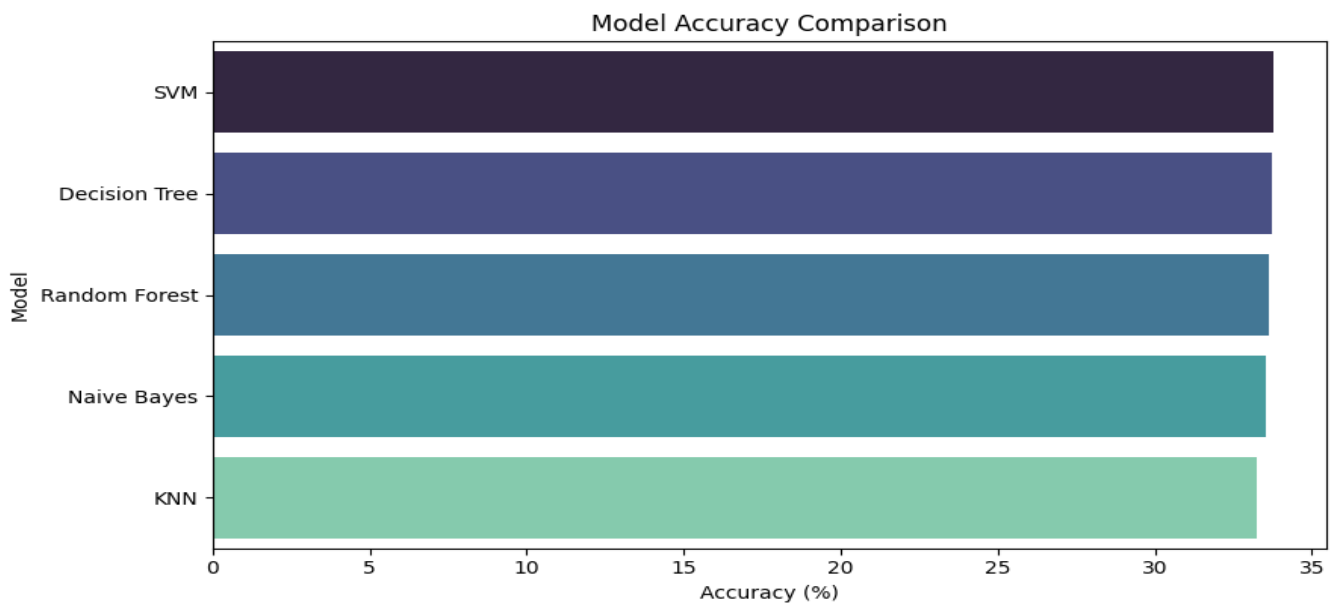
Confusion Matrix (Random Forest):

[[1482 1241 1276]
[1517 1271 1290]
[1459 1179 1285]]

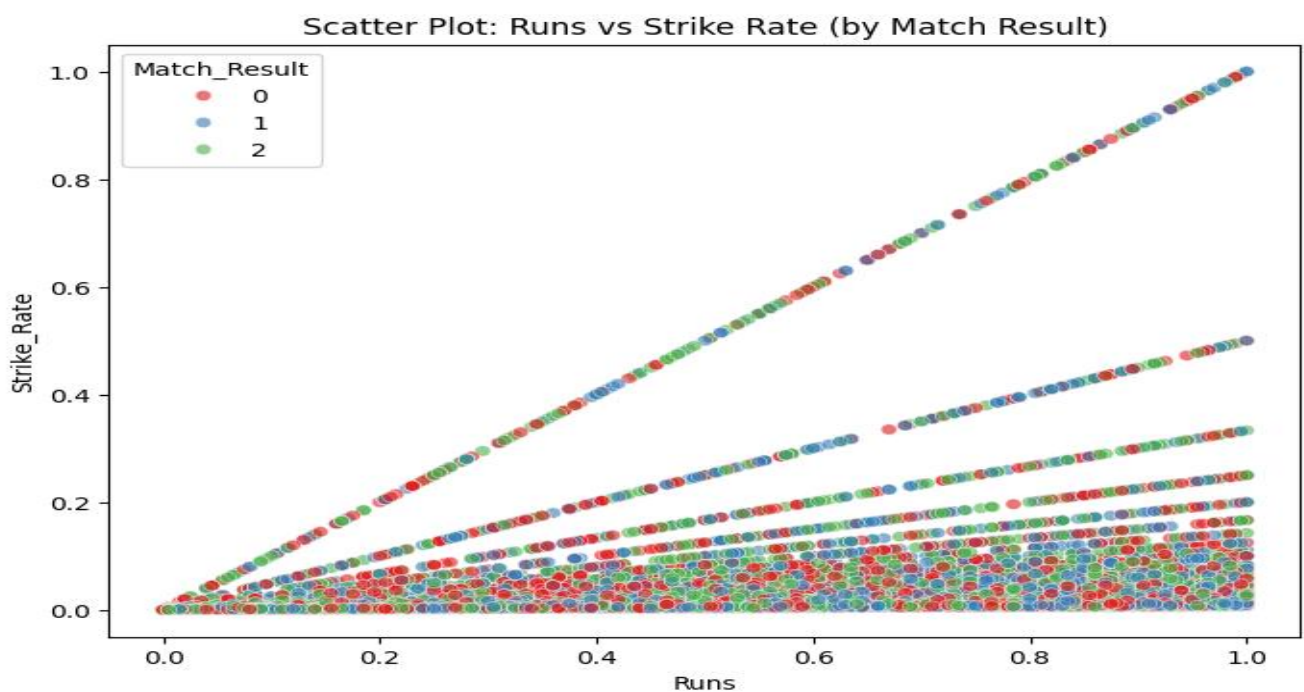
Accuracy (Random Forest): 33.65 %

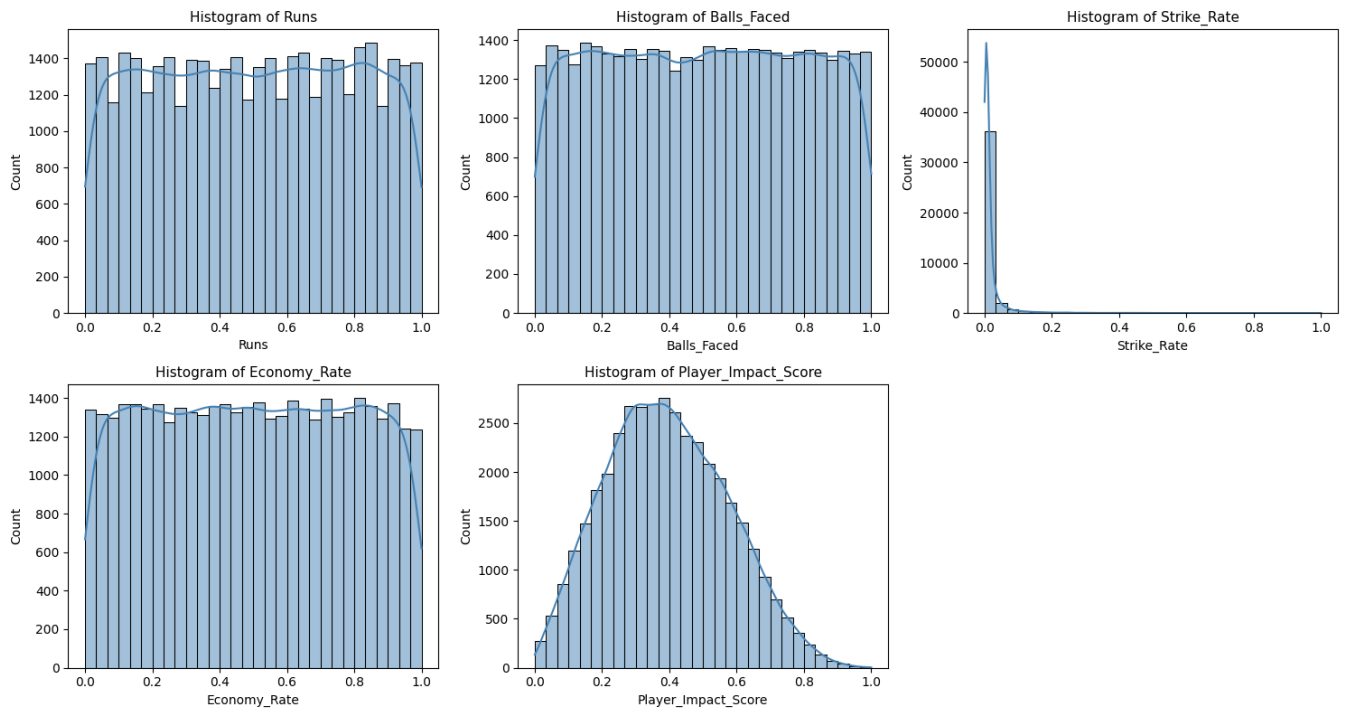
Classification Report (Random Forest):

	precision	recall	f1-score	support
0	0.33	0.37	0.35	3999
1	0.34	0.31	0.33	4078
2	0.33	0.33	0.33	3923
accuracy			0.34	12000
macro avg	0.34	0.34	0.34	12000
weighted avg	0.34	0.34	0.34	12000

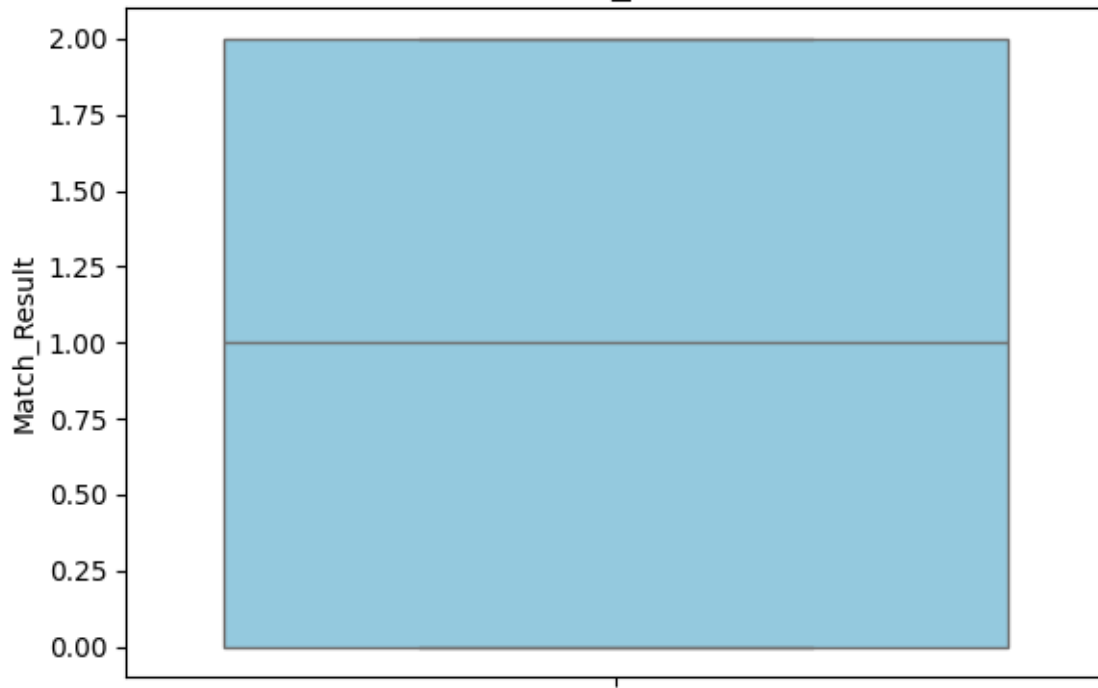


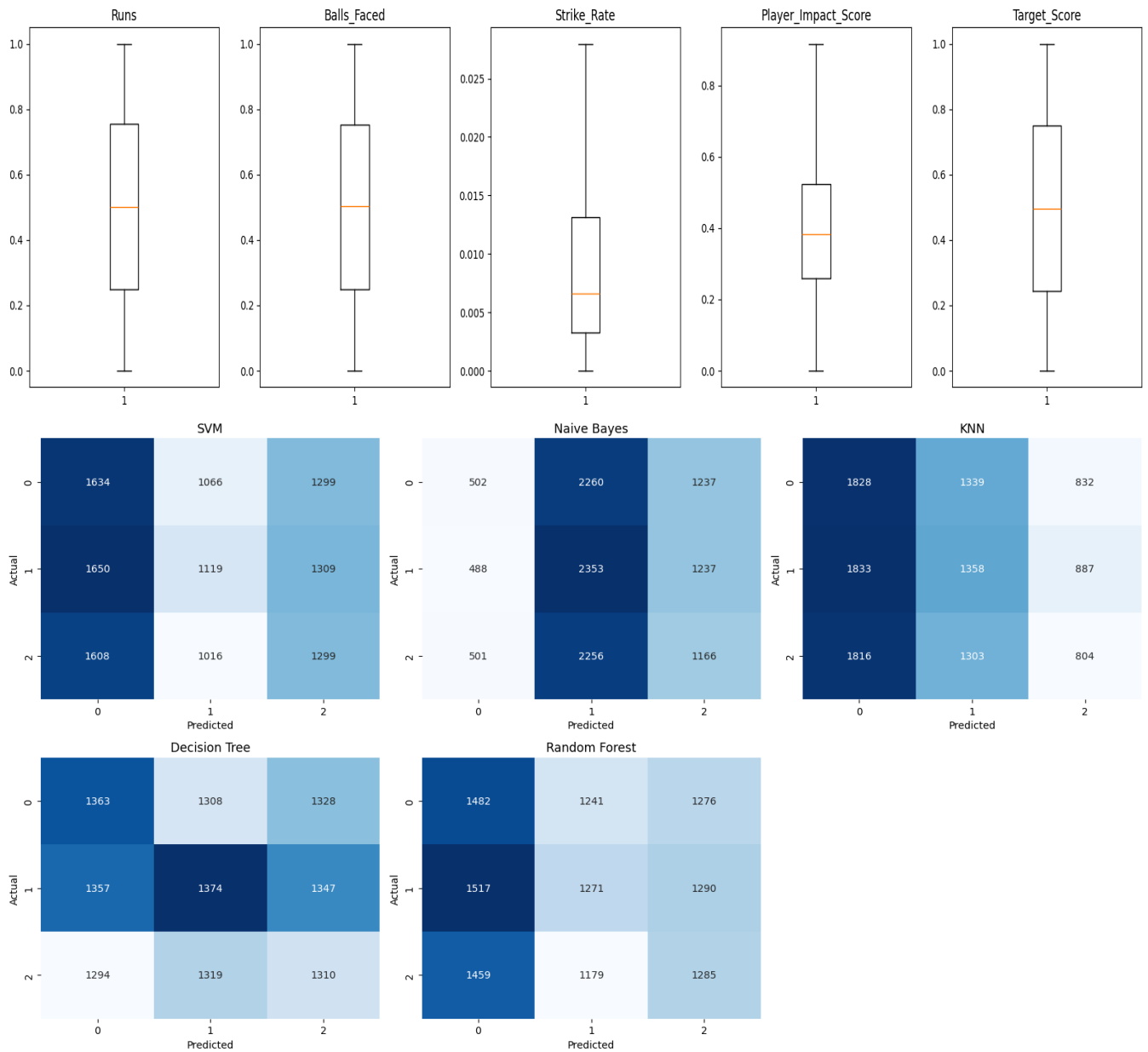
Best Model: SVM





Box Plot of Match_Result (encoded)





4.2 Test Report :

Each model was evaluated using Confusion Matrix, Accuracy Score, and Classification Report. The classification reports include Precision, Recall, F1-score, and Support.

- **Support Vector Machine (SVM):** Accuracy – **33.77%**
- **Naïve Bayes:** Accuracy – **33.51%**
- **K-Nearest Neighbors (KNN):** Accuracy – **33.25%**
- **Decision Tree:** Accuracy – **33.73%**
- **Random Forest:** Accuracy – **33.65%**

5. Proposed Enhancements:

The system can be improved by integrating real-time match data, advanced machine learning models, interactive dashboards, cloud deployment, and mobile applications. Additional features like weather analysis, player fitness tracking, and fantasy team recommendations can further enhance prediction accuracy and usability.

6. Conclusion :

The Cricket Player Performance Analysis system successfully analyzes player statistics using data science and machine learning techniques. It helps coaches, selectors, and analysts make better decisions by providing accurate performance insights through data analysis and visualization.

7. Bibliography :

1. Scikit-learn: Machine Learning in Python – <https://scikit-learn.org/>
2. Pandas Documentation – <https://pandas.pydata.org/>
3. NumPy Documentation – <https://numpy.org/>
4. Matplotlib Documentation – <https://matplotlib.org/>
5. Seaborn Documentation – <https://seaborn.pydata.org/>
6. Kaggle Cricket Dataset – <https://www.kaggle.com/>
7. ESPN Cricinfo Statistics – <https://www.espncricinfo.com/>
8. ICC Official Website – <https://www.icc-cricket.com/>